

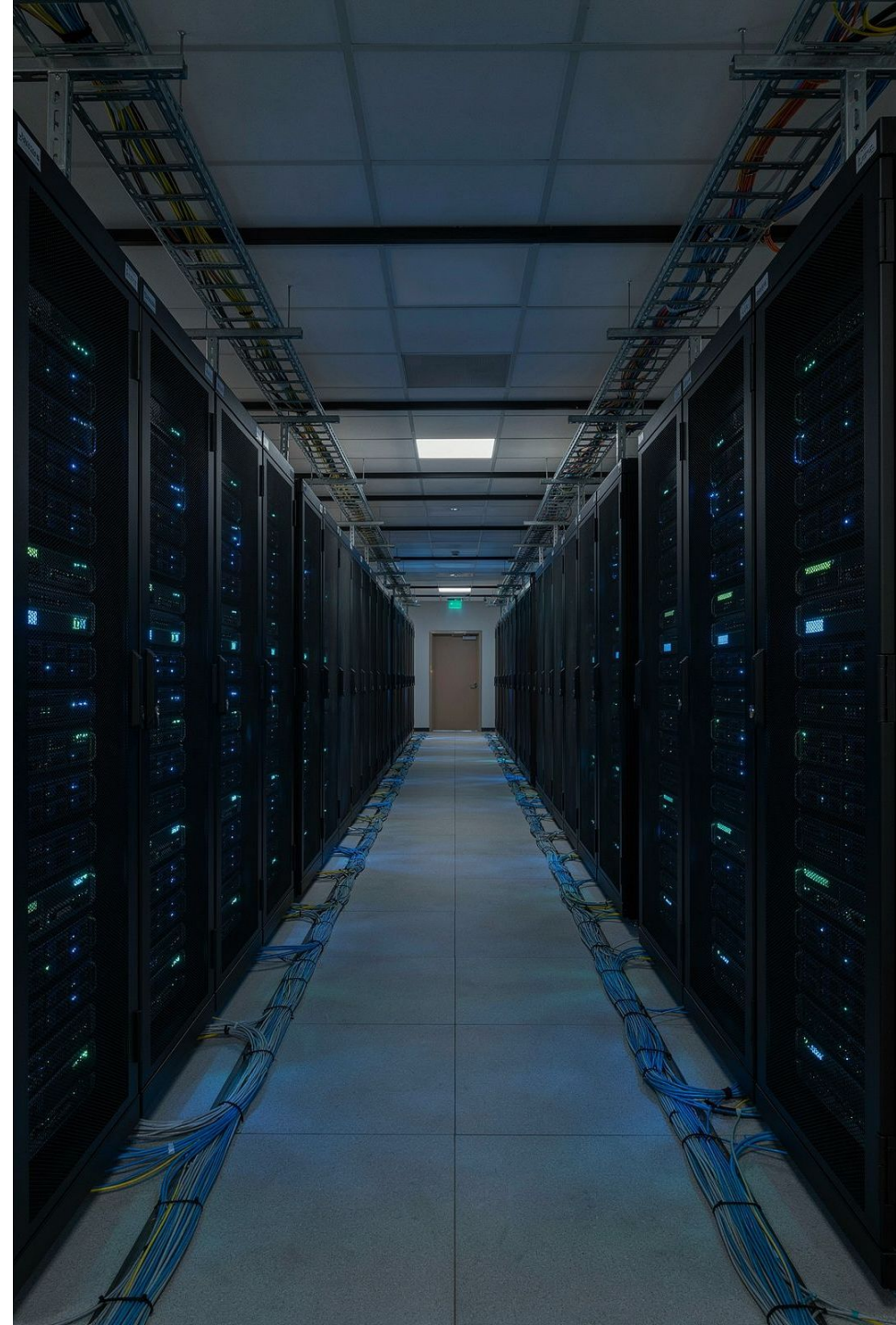
Bases de Datos Relacionales y SQL

Módulo S3.1 — Una guía completa desde los fundamentos del modelo relacional hasta las buenas prácticas de administración, pensada para proyectos de inteligencia artificial y desarrollo de software profesional.

S3.1

SQL

BASES DE DATOS



Lo que aprenderemos hoy

Este módulo cubre el ciclo completo de trabajo con bases de datos relacionales: desde su papel en proyectos de IA hasta la administración avanzada y las buenas prácticas SQL.

01

Fundamentos

Modelo relacional, tablas, claves y relaciones

02

Diseño

Normalización y estructura de esquemas

03

SQL

DDL, CRUD, consultas, JOINS y agregaciones

04

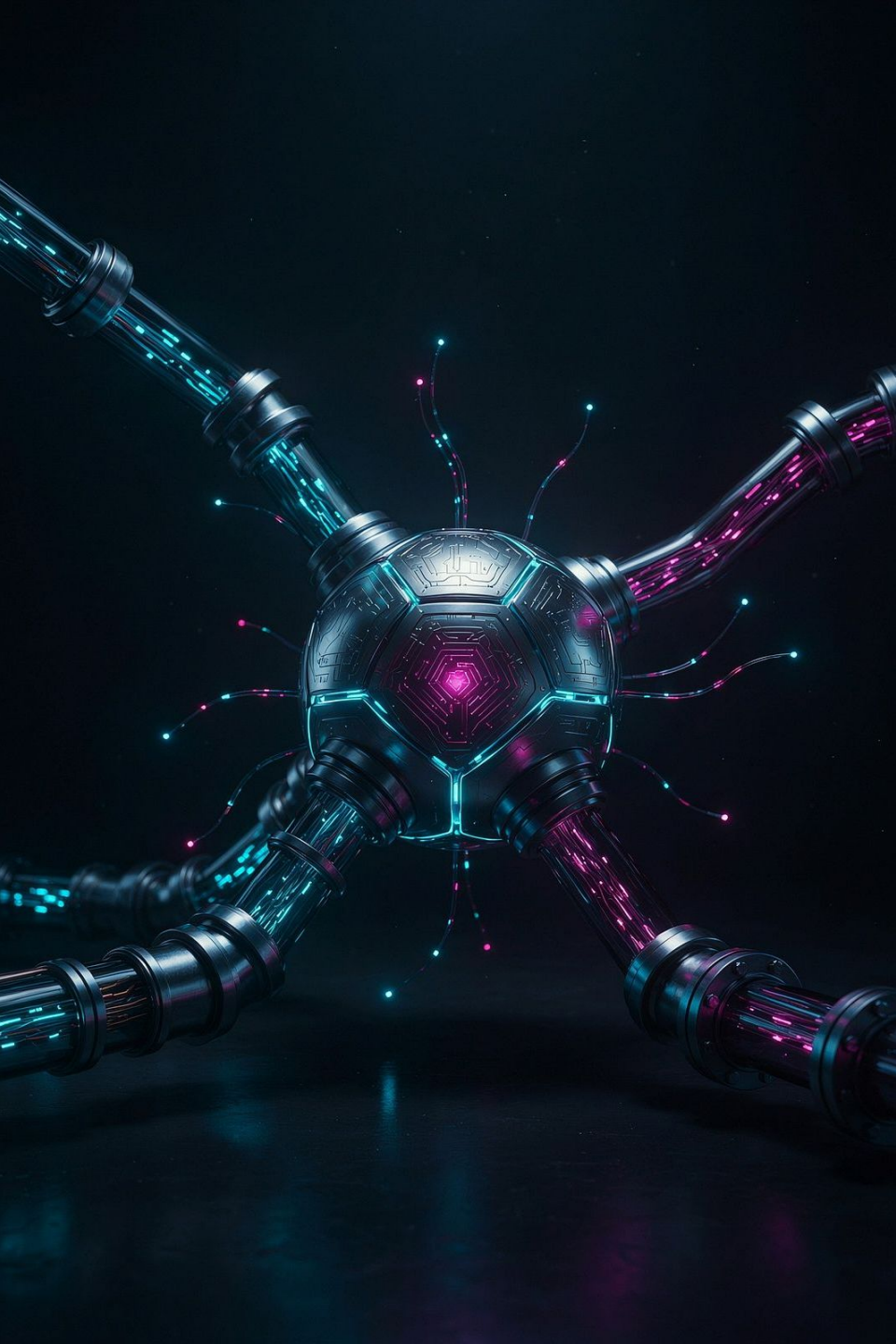
Administración

Índices, ACID, usuarios y permisos

05

Laboratorio

Práctica guiada con casos reales



¿Dónde Encajan las Bases de Datos en un Proyecto de IA?

Las bases de datos no son un accesorio opcional: son la columna vertebral de cualquier sistema de inteligencia artificial. Almacenan los datos de entrenamiento, registran predicciones, gestionan usuarios y garantizan la reproducibilidad de los experimentos.

El Papel de las Bases de Datos en la IA



Almacenamiento de Datos

Guardan conjuntos de datos estructurados listos para el preprocesamiento y el entrenamiento de modelos.



Registro de Experimentos

Persisten métricas, hiperparámetros y versiones de modelos para comparar y reproducir resultados.



Gestión de Usuarios

Controlan el acceso a datos sensibles mediante roles y permisos granulares.



Integración con APIs

Sirven datos en tiempo real a las aplicaciones que consumen los modelos de IA en producción.

Flujo de Datos en un Pipeline de IA



La base de datos relacional actúa como capa central de confianza: normaliza, versiona y sirve los datos a todas las fases del pipeline con consistencia y trazabilidad.

El Modelo Relacional

Tablas, claves y relaciones — los tres pilares que hacen de las bases de datos relacionales el estándar de la industria durante más de cuatro décadas.



Conceptos Fundamentales del Modelo Relacional

¿Qué es una tabla?

Una tabla (o relación) organiza datos en filas y columnas. Cada **fila** representa una entidad individual (registro) y cada **columna** un atributo con un tipo de dato definido.

Ejemplo: la tabla `clientes` tiene columnas como `id`, `nombre`, `email` y `fecha_alta`.

Propiedades esenciales

- Cada celda contiene un único valor atómico
- El orden de las filas no tiene significado
- Cada columna tiene un nombre único dentro de la tabla
- Los datos de una columna son del mismo tipo

📄 El modelo relacional fue propuesto por Edgar F. Codd en 1970 y sigue siendo el paradigma dominante en sistemas transaccionales.

Claves: Identidad y Relaciones entre Tablas



Clave Primaria (PRIMARY KEY)

Identifica de forma única cada fila de una tabla. No admite valores nulos ni duplicados.

Ejemplo: `id INTEGER PRIMARY KEY`. Puede ser simple (un campo) o compuesta (varios campos combinados).



Clave Foránea (FOREIGN KEY)

Referencia la clave primaria de otra tabla, estableciendo una relación entre entidades.

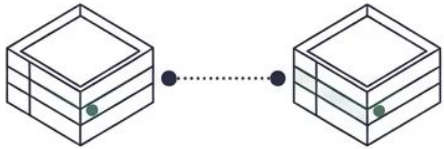
Garantiza la **integridad referencial**: no se puede insertar un valor que no exista en la tabla padre.



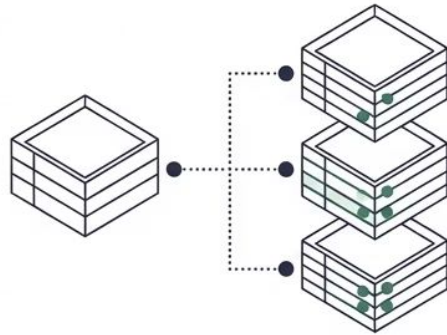
Clave Única (UNIQUE)

Garantiza que todos los valores de una columna son distintos, sin necesidad de ser la clave primaria. Útil para campos como `email` o `número_de_serie`.

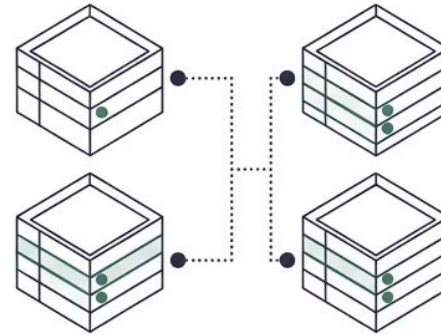
Tipos de Relaciones entre Tablas



Uno a Uno (1:1)
— un pasaporte
por persona



Uno a Muchos (1:N) — un cliente
tiene varios
pedidos



Muchos a Muchos (M:N) —
estudiantes
matriculados en
varias asignaturas
y asignaturas con
varios estudiantes

Las relaciones **M:N** se implementan mediante una tabla intermedia (tabla de unión) que contiene las claves foráneas de ambas entidades participantes.

Diseño y Normalización

Un buen diseño de base de datos elimina redundancias, previene anomalías y garantiza la integridad de los datos a largo plazo.



¿Qué es la Normalización?

La normalización es el proceso de organizar las tablas y columnas de una base de datos para **reducir la redundancia** y mejorar la integridad de los datos. Se aplica mediante una serie de reglas llamadas **formas normales**.

Una base de datos mal normalizada puede presentar tres tipos de anomalías:

- **Inserción:** no se puede añadir un dato sin otro relacionado
- **Actualización:** cambiar un dato requiere múltiples actualizaciones
- **Eliminación:** borrar un registro elimina información no deseada

1FN

Valores atómicos,
sin grupos
repetidos

2FN

Sin dependencias
parciales de la PK

3FN

Sin dependencias transitivas

Las Tres Primeras Formas Normales

1

Primera Forma Normal (1FN)

Cada celda contiene un único valor atómico. No hay columnas repetidas ni conjuntos de valores en un solo campo.

2

Segunda Forma Normal (2FN)

Cumple 1FN y todos los atributos no clave dependen completamente de la clave primaria completa (elimina dependencias parciales).

3

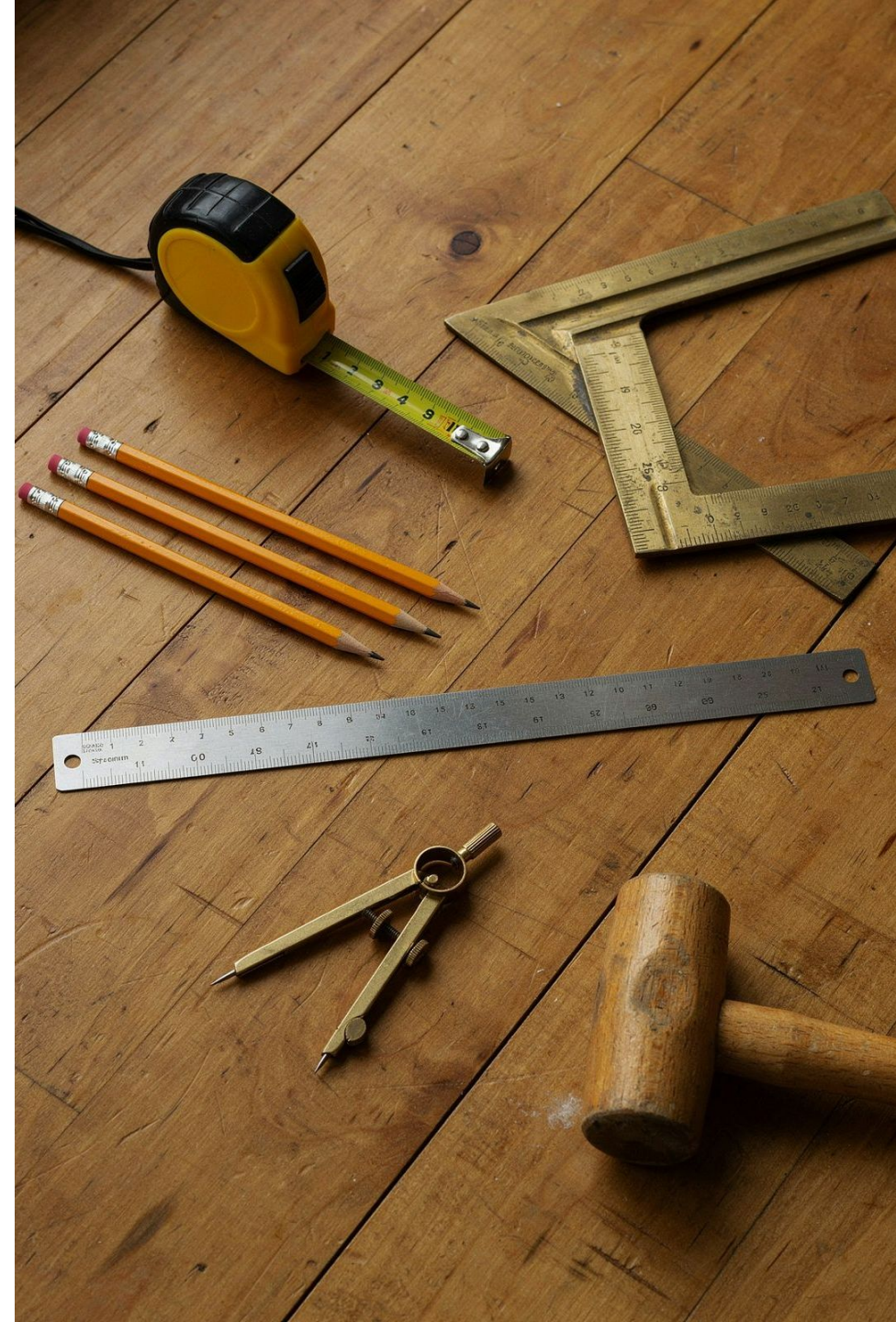
Tercera Forma Normal (3FN)

Cumple 2FN y ningún atributo no clave depende de otro atributo no clave (elimina dependencias transitivas).

- ❏ En la práctica, llegar a 3FN es suficiente para la mayoría de los sistemas. La desnormalización controlada puede ser útil en escenarios de alto rendimiento de lectura.

DDL: Crear y Modificar Tablas

El lenguaje de definición de datos (DDL) permite crear, modificar y eliminar la estructura de las bases de datos. Es la base sobre la que se construye todo el sistema.



Comandos DDL Fundamentales



CREATE TABLE

Define una nueva tabla con sus columnas, tipos de datos y restricciones (PK, FK, NOT NULL, UNIQUE, DEFAULT).



ALTER TABLE

Modifica una tabla existente: añadir o eliminar columnas, cambiar tipos de datos o agregar restricciones sin recrear la tabla.



DROP TABLE

Elimina una tabla y todos sus datos de forma permanente. Usar con precaución en entornos de producción.



TRUNCATE

Vacía todos los registros de una tabla de forma rápida, conservando su estructura y restricciones.

Ejemplo: Crear una Tabla con DDL

```
CREATE TABLE clientes (  
  id    INTEGER    PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  pais  CHAR(2)    DEFAULT 'ES',  
  fecha_alta DATE    DEFAULT CURRENT_DATE  
);  
  
-- Añadir una columna posterior  
ALTER TABLE clientes  
  ADD COLUMN telefono VARCHAR(20);  
  
-- Eliminar la tabla (¡cuidado!)  
DROP TABLE IF EXISTS clientes;
```

Tipos de datos más comunes

- INTEGER / BIGINT — números enteros
- DECIMAL(p,s) — números decimales precisos
- VARCHAR(n) — texto de longitud variable
- TEXT — texto sin límite de longitud
- DATE / TIMESTAMP — fechas y horas
- BOOLEAN — verdadero / falso
- JSONB — datos semiestructurados (PostgreSQL)

CRUD: Las Cuatro Operaciones Esenciales

INSERT, SELECT, UPDATE y DELETE son las operaciones fundamentales de manipulación de datos. Todo sistema de información, desde una app móvil hasta un pipeline de ML, las utiliza constantemente.



INSERT: Añadir Datos

-- Insertar un único registro

```
INSERT INTO clientes (nombre, email, pais)
VALUES ('Ana García', 'ana@ejemplo.es', 'ES');
```

-- Insertar múltiples registros

```
INSERT INTO clientes (nombre, email, pais)
VALUES
('Luis Martín', 'luis@ejemplo.es', 'ES'),
('Marie Dupont', 'marie@ejemplo.fr', 'FR'),
('João Silva', 'joao@ejemplo.pt', 'PT');
```

-- Insertar desde otra tabla

```
INSERT INTO clientes_backup
SELECT * FROM clientes
WHERE fecha_alta < '2024-01-01';
```

Buenas prácticas al insertar

- Especifica siempre la lista de columnas, nunca asumas un orden
- Valida los datos antes de insertarlos desde la aplicación
- Usa `INSERT ... ON CONFLICT (upsert)` para evitar duplicados
- Agrupa múltiples inserciones en un solo comando para mejorar el rendimiento

SELECT: Consultar Datos

-- Seleccionar todas las columnas

```
SELECT * FROM clientes;
```

-- Seleccionar columnas específicas

```
SELECT nombre, email FROM clientes;
```

-- Alias de columnas

```
SELECT
```

```
  nombre      AS cliente,
```

```
  fecha_alta  AS "Fecha de registro"
```

```
FROM clientes;
```

-- Eliminar duplicados

```
SELECT DISTINCT pais FROM clientes;
```

La anatomía de un SELECT

Un SELECT completo sigue este orden lógico:

1. **SELECT** — qué columnas mostrar
2. **FROM** — de qué tabla
3. **WHERE** — qué filas filtrar
4. **GROUP BY** — cómo agrupar
5. **HAVING** — filtrar grupos
6. **ORDER BY** — cómo ordenar
7. **LIMIT** — cuántas filas devolver

UPDATE y DELETE: Modificar y Eliminar

-- Actualizar un registro específico

```
UPDATE clientes
SET email = 'ana.garcia@nuevo.es',
    pais = 'MX'
WHERE id = 1;
```

-- Actualización masiva controlada

```
UPDATE clientes
SET pais = 'ES'
WHERE pais IS NULL;
```

-- Eliminar un registro

```
DELETE FROM clientes
WHERE id = 42;
```

-- Eliminar registros con condición

```
DELETE FROM clientes
WHERE fecha_alta < '2020-01-01'
AND pais = 'XX';
```



Reglas de oro

- Nunca ejecute un UPDATE o DELETE sin cláusula WHERE en producción
- Pruebe primero con un SELECT equivalente para ver qué filas se verán afectadas
- Use transacciones (BEGIN / ROLLBACK) para poder deshacer cambios accidentales
- Considere el borrado lógico (deleted_at) en lugar del físico para datos críticos

Consultas Avanzadas: WHERE, ORDER BY y LIMIT

Filtrar, ordenar y paginar son las tres habilidades que convierten un conjunto de millones de registros en información relevante y accionable.



Filtrado con WHERE

-- Operadores de comparación

```
SELECT * FROM pedidos  
WHERE total > 100 AND estado = 'enviado';
```

-- Rango de valores

```
SELECT * FROM productos  
WHERE precio BETWEEN 10 AND 50;
```

-- Lista de valores

```
SELECT * FROM clientes  
WHERE pais IN ('ES', 'MX', 'AR');
```

-- Búsqueda de texto con patrones

```
SELECT * FROM clientes  
WHERE nombre LIKE 'García%';
```

-- Valores nulos

```
SELECT * FROM clientes  
WHERE telefono IS NULL;
```

Operadores lógicos

AND

Ambas condiciones deben cumplirse

OR

Al menos una condición se cumple

NOT

Niega la condición siguiente

Usa paréntesis para controlar la precedencia: `WHERE (a OR b) AND c`

Ordenar y Pagar Resultados

-- Ordenar por una columna (ascendente por defecto)

```
SELECT nombre, fecha_alta  
FROM clientes  
ORDER BY fecha_alta DESC;
```

-- Ordenar por múltiples columnas

```
SELECT nombre, pais, total  
FROM pedidos  
ORDER BY pais ASC, total DESC;
```

-- Paginación con LIMIT y OFFSET

```
SELECT nombre, email  
FROM clientes  
ORDER BY id  
LIMIT 20 OFFSET 40; -- página 3 de 20 en 20
```

Patrones de paginación

La paginación con **OFFSET** es sencilla pero ineficiente en tablas grandes. Para millones de registros, usa **keyset pagination**:

```
SELECT * FROM clientes  
WHERE id > :ultimo_id  
ORDER BY id  
LIMIT 20;
```

Este patrón es constante en tiempo independientemente de la página, ideal para feeds y APIs de alto volumen.

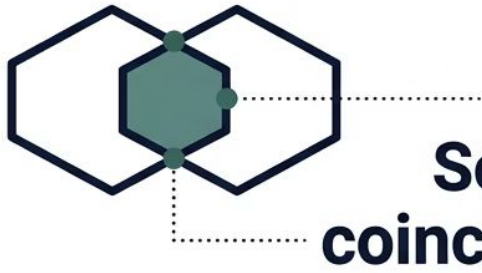
JOIN: Combinar Información de Múltiples Tablas

Los JOINS son la operación más poderosa del modelo relacional. Permiten reconstruir información distribuida en diferentes tablas en una única consulta coherente.



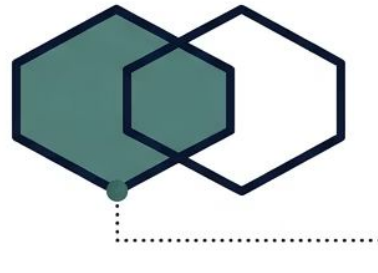
Tipos de JOIN

INNER JOIN



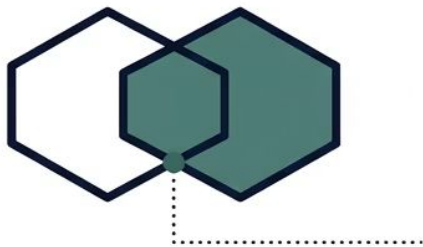
**Sólo filas
coincidentes**

LEFT JOIN



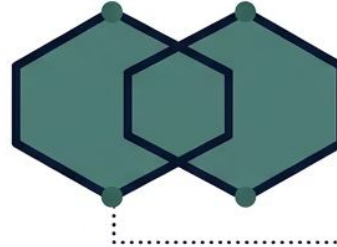
**Todas
las filas
izquierdas**

RIGHT JOIN



**Todas
las filas
derechas**

FULL OUTER JOIN



**Todas
las filas**

El **INNER JOIN** es el más utilizado y devuelve únicamente las filas con correspondencia en ambas tablas. El **LEFT JOIN** es esencial para detectar registros huérfanos.

JOIN en la Práctica

-- INNER JOIN: pedidos con datos del cliente

```
SELECT
  c.nombre,
  p.fecha,
  p.total
FROM pedidos p
INNER JOIN clientes c ON p.cliente_id = c.id
WHERE p.total > 200;
```

-- LEFT JOIN: clientes sin ningún pedido

```
SELECT c.nombre, p.id AS pedido_id
FROM clientes c
LEFT JOIN pedidos p ON c.id = p.cliente_id
WHERE p.id IS NULL;
```

-- Múltiples JOINs encadenados

```
SELECT c.nombre, pr.nombre AS producto, l.cantidad
FROM pedidos p
JOIN clientes c ON p.cliente_id = c.id
JOIN lineas l ON l.pedido_id = p.id
JOIN productos pr ON l.producto_id = pr.id;
```

Consejos para JOINs eficientes

- Usa siempre **alias** de tabla para mayor legibilidad
- Asegúrate de que las columnas de unión tienen **índices**
- Filtra antes del JOIN cuando sea posible para reducir filas
- Evita el `SELECT *` en consultas con múltiples JOINs para no traer columnas innecesarias
- El **producto cartesiano** (sin ON) rara vez es lo que quieres — verifica siempre la condición de unión

A wooden bookshelf with four shelves. The top shelf is filled with books of various colors (red, green, blue, brown). The second shelf has books organized by subject, with labels 'SCIENCE', 'ARTS', 'MUSIC', and 'PHILOSOPHY' placed below them. The third shelf is filled with manila-colored folders, labeled 'A-C', 'D-F', 'G-I', and 'J-L'. The bottom shelf has binders labeled 'ARCHIVES', 'RECORDS', and 'JOURNALS', along with more books.

CAPÍTULO 7

GROUP BY y Funciones de Agregación

Las agregaciones transforman filas individuales en resúmenes estadísticos. Son la base de los informes, cuadros de mando y análisis exploratorio de datos.

Funciones de Agregación Principales

COUNT()

Cuenta filas o valores no nulos. `COUNT(*)` cuenta todas las filas; `COUNT(columna)` excluye nulos.

SUM()

Suma los valores numéricos de un grupo. Ideal para totales de ventas, importes y cantidades.

AVG()

Calcula la media aritmética. Ignora valores nulos automáticamente.

MIN / MAX

Devuelven el valor mínimo y máximo del grupo. Funcionan también con fechas y texto.

GROUP BY y HAVING en Acción

-- Ventas totales por país

```
SELECT
  pais,
  COUNT(*)    AS total_clientes,
  SUM(p.total) AS ingresos_totales,
  AVG(p.total) AS ticket_medio
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
GROUP BY pais
ORDER BY ingresos_totales DESC;
```

-- HAVING: filtrar grupos (no filas)

```
SELECT pais, COUNT(*) AS clientes
FROM clientes
GROUP BY pais
HAVING COUNT(*) > 50
ORDER BY clientes DESC;
```

WHERE vs. HAVING

WHERE filtra **filas individuales** antes de agrupar.

HAVING filtra **grupos después** de calcular la agregación.

Ejemplo: para obtener países con más de 50 clientes registrados en 2024, usa WHERE para el año y HAVING para el conteo mínimo. No se puede usar `WHERE COUNT(*) > 50` — eso es un error de sintaxis.

Nociones de Administración de Bases de Datos

Un DBA (Database Administrator) garantiza que la base de datos sea rápida, segura, consistente y recuperable. Estos principios son esenciales incluso para desarrolladores que no son DBAs de oficio.



Índices: Acelerar las Consultas

Un **índice** es una estructura de datos auxiliar que permite localizar filas sin escanear toda la tabla. Funciona de forma similar al índice de un libro: en lugar de leer página por página, saltas directamente a la entrada correcta.

-- Crear un índice simple

```
CREATE INDEX idx_clientes_pais  
ON clientes (pais);
```

-- Índice compuesto

```
CREATE INDEX idx_pedidos_cliente_fecha  
ON pedidos (cliente_id, fecha);
```

-- Índice único

```
CREATE UNIQUE INDEX idx_email  
ON clientes (email);
```

Cuándo usar índices

- Columnas usadas frecuentemente en **WHERE**, **JOIN** y **ORDER BY**
- Claves foráneas (evitan escaneos completos en JOINS)
- Columnas con alta cardinalidad (muchos valores distintos)

Cuándo no usarlos

- Tablas muy pequeñas (el escaneo completo es más rápido)
- Columnas que se actualizan constantemente (coste de mantenimiento)
- Tablas con cargas masivas de escritura

Transacciones y las Propiedades ACID



Atomicidad

Una transacción es todo o nada. Si falla cualquier paso, se deshace todo automáticamente con `ROLLBACK`.



Consistencia

La base de datos pasa de un estado válido a otro. Nunca viola restricciones de integridad durante la transacción.



Aislamiento

Las transacciones concurrentes no se interfieren entre sí. Cada una ve la base de datos como si fuera la única ejecutándose.



Durabilidad

Una vez confirmada (`COMMIT`), la transacción persiste aunque el sistema se caiga inmediatamente después.

Transacciones en SQL

-- Transferencia bancaria segura

BEGIN;

UPDATE cuentas

SET saldo = saldo - 500

WHERE id = 1;

UPDATE cuentas

SET saldo = saldo + 500

WHERE id = 2;

-- Si todo fue bien

COMMIT;

-- Si algo falló, deshacer todo

ROLLBACK;

Puntos de guardado (Savepoints)

Los savepoints permiten deshacer parcialmente una transacción sin perder todo el trabajo:

BEGIN;

INSERT INTO pedidos ...;

SAVEPOINT sp1;

INSERT INTO lineas ...;

-- Solo deshacemos desde sp1

ROLLBACK TO sp1;

COMMIT;

Útiles en transacciones largas o complejas con múltiples pasos independientes.

Copias de Seguridad, Usuarios y Permisos



Copias de Seguridad

Los backups lógicos

(`pg_dump`, `mysqldump`)

exportan el esquema y los datos. Los físicos copian los ficheros del sistema.

Automatiza las copias y verifica periódicamente su restauración.



Gestión de Usuarios

Crea un usuario por aplicación o servicio, nunca uses el superusuario (`root` / `postgres`) en producción. Los usuarios se crean con `CREATE USER` y se borran con `DROP USER`.



Permisos con GRANT y REVOKE

Aplica el **principio de mínimo privilegio**: cada usuario solo debe tener los permisos estrictamente necesarios. Usa `GRANT SELECT ON tabla TO usuario` y `REVOKE` para retirarlos.

Buenas Prácticas SQL

El código SQL de calidad es legible, mantenible, eficiente y seguro. Estas prácticas marcan la diferencia entre un sistema robusto y uno que genera deuda técnica.



Buenas Prácticas de Escritura SQL

Legibilidad y Mantenimiento

- Escribe las palabras clave en mayúsculas (`SELECT`, `FROM`, `WHERE`)
- Indenta subconsultas y condiciones complejas
- Usa alias descriptivos en tablas y columnas calculadas
- Divide consultas muy largas en CTEs (`WITH`)
- Comenta el propósito de las consultas complejas

Seguridad y Rendimiento

- Usa **consultas parametrizadas** para prevenir inyección SQL
- Evita `SELECT *` en producción — selecciona solo las columnas necesarias
- Analiza el plan de ejecución con `EXPLAIN ANALYZE` antes de optimizar
- Limita los resultados con `LIMIT` en consultas exploratorias
- Revisa los índices cuando una consulta sea más lenta de lo esperado

CTEs: Consultas Legibles y Reutilizables

```
-- Common Table Expression (WITH)
WITH ventas_por_pais AS (
  SELECT
    c.pais,
    SUM(p.total) AS total_ventas
  FROM clientes c
  JOIN pedidos p ON c.id = p.cliente_id
  GROUP BY c.pais
),
media_global AS (
  SELECT AVG(total_ventas) AS media
  FROM ventas_por_pais
)
SELECT
  v.pais,
  v.total_ventas,
  ROUND(v.total_ventas / m.media * 100, 1)
  AS pct_sobre_media
FROM ventas_por_pais v
CROSS JOIN media_global m
ORDER BY v.total_ventas DESC;
```

¿Por qué usar CTEs?

- Dividen consultas complejas en bloques nombrados y comprensibles
- Evitan la repetición de subconsultas idénticas
- Facilitan el testing incremental paso a paso
- Los CTEs recursivos permiten recorrer jerarquías (árboles, grafos)

📌 Los CTEs son equivalentes a vistas temporales dentro de una consulta. En PostgreSQL 12+ el optimizador los integra automáticamente en el plan de ejecución.

Prevención de Inyección SQL

✗ Código vulnerable

```
# Python — NUNCA hagas esto
nombre = request.args.get('nombre')
query = f"SELECT * FROM clientes
        WHERE nombre = '{nombre}'"
cursor.execute(query)
# Un atacante puede pasar:
# ' OR '1'='1 → devuelve TODOS los registros
# '; DROP TABLE clientes; -- → destruye la tabla
```

✓ Código seguro con parámetros

```
# Python con psycopg2 — forma correcta
nombre = request.args.get('nombre')
query = """
        SELECT * FROM clientes
        WHERE nombre = %s
        """
cursor.execute(query, (nombre,))
# El driver escapa automáticamente el input
# Un atacante no puede inyectar SQL malicioso
```

La inyección SQL es una de las vulnerabilidades más comunes y peligrosas. Siempre usa **consultas parametrizadas** o un ORM que las gestione automáticamente.

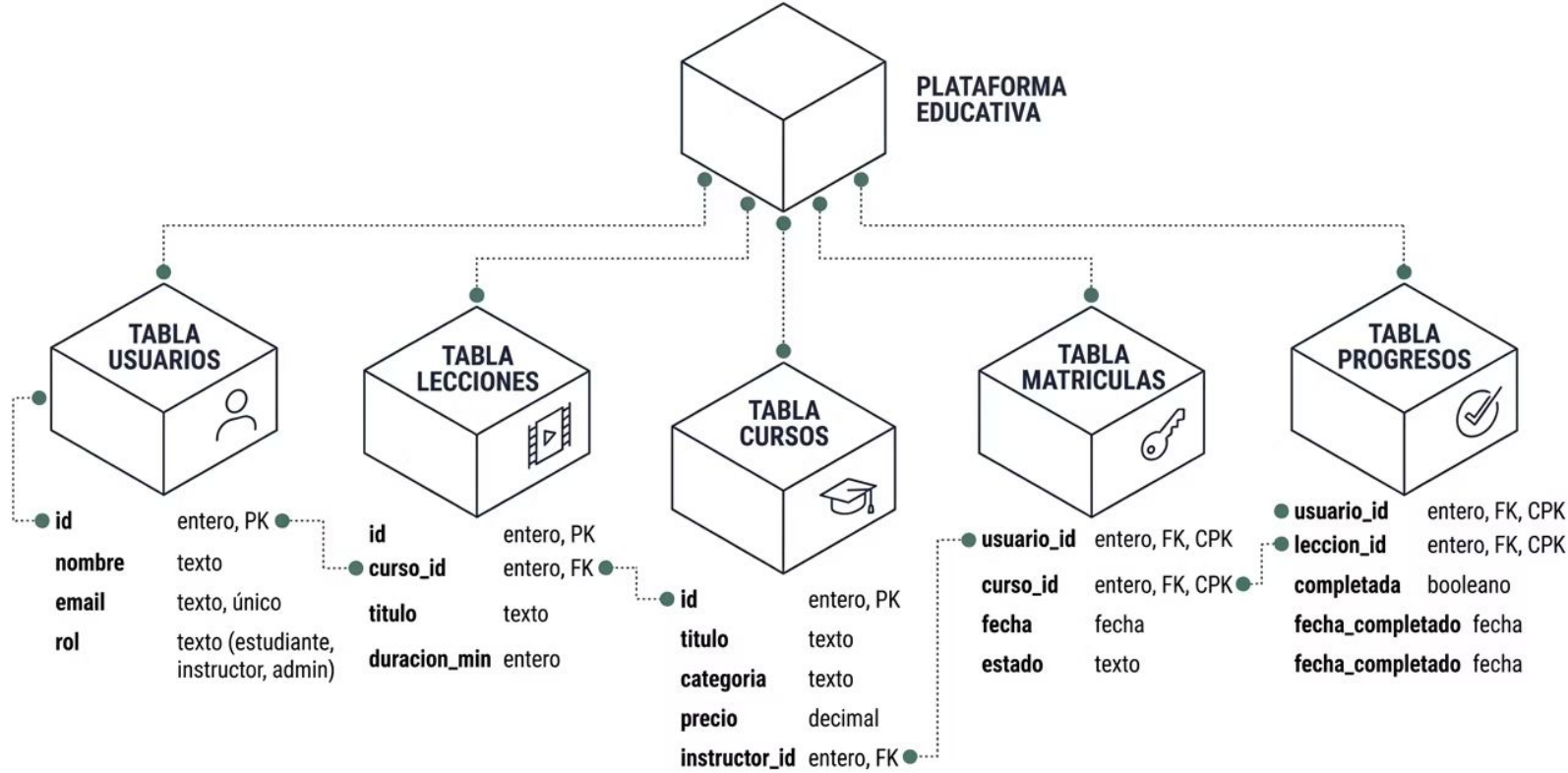
Laboratorio Guiado

Ponemos en práctica todos los conceptos del módulo construyendo una base de datos desde cero para un sistema de gestión de cursos online.



Caso Práctico: Sistema de Cursos Online

Diseñaremos e implementaremos una base de datos para una plataforma educativa con las siguientes entidades:



Paso 1: Crear el Esquema

```
CREATE TABLE usuarios (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  rol VARCHAR(20) DEFAULT 'estudiante',  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE cursos (  
  id SERIAL PRIMARY KEY,  
  titulo VARCHAR(200) NOT NULL,  
  categoria VARCHAR(50),  
  precio DECIMAL(8,2) DEFAULT 0,  
  instructor_id INTEGER REFERENCES usuarios(id)  
);
```

```
CREATE TABLE matriculas (  
  usuario_id INTEGER REFERENCES usuarios(id),  
  curso_id INTEGER REFERENCES cursos(id),  
  fecha DATE DEFAULT CURRENT_DATE,  
  estado VARCHAR(20) DEFAULT 'activa',  
  PRIMARY KEY (usuario_id, curso_id)  
);
```

Paso 2: Poblar e Interrogar los Datos

-- Insertar datos de prueba

```
INSERT INTO usuarios (nombre, email, rol) VALUES
('Carlos Pérez', 'carlos@edu.es', 'instructor'),
('Laura Gómez', 'laura@edu.es', 'estudiante'),
('Marcos Ruiz', 'marcos@edu.es', 'estudiante');
```

```
INSERT INTO cursos (titulo, categoria, precio, instructor_id)
VALUES ('SQL desde cero', 'Bases de datos', 49.99, 1);
```

```
INSERT INTO matriculas VALUES (2, 1, DEFAULT, DEFAULT),
(3, 1, DEFAULT, DEFAULT);
```

-- Consultas de análisis

-- ¿Cuántos estudiantes por curso?

```
SELECT
c.titulo,
COUNT(m.usuario_id) AS matriculados
FROM cursos c
LEFT JOIN matriculas m ON c.id = m.curso_id
GROUP BY c.titulo
ORDER BY matriculados DESC;
```

-- Cursos con más de 100 matriculados

```
SELECT titulo, COUNT(*) AS n
FROM cursos c JOIN matriculas m ON c.id=m.curso_id
GROUP BY titulo
HAVING COUNT(*) > 100;
```

Resumen del Módulo S3.1

1 Modelo relacional

Tablas, claves primarias, foráneas y tipos de relaciones (1:1, 1:N, M:N) son los pilares del diseño de bases de datos.

3 SQL completo

DDL para definir estructuras, CRUD para manipular datos, JOINS para combinar tablas y GROUP BY para analizar.

2 Diseño y normalización

Las tres formas normales eliminan redundancias y previenen anomalías de inserción, actualización y eliminación.

4 Administración y seguridad

Índices, ACID, backups, usuarios con mínimo privilegio y consultas parametrizadas son imprescindibles en producción.



Próximos Pasos

Practica SQL

Plataformas como SQLZoo, LeetCode o Mode Analytics ofrecen ejercicios progresivos con bases de datos reales.

Aprende PostgreSQL

PostgreSQL es el motor open-source más completo y el más usado en proyectos de IA y datos. Domina sus extensiones como `pgvector` para embeddings.

Explora ORMs

SQLAlchemy (Python) y Prisma (Node.js) generan SQL seguro y parametrizado automáticamente, acelerando el desarrollo.

Módulo S3.2

En el siguiente módulo exploraremos bases de datos NoSQL, vectoriales y su integración con modelos de lenguaje.